A CALM COMMAND CENTER FOR WORK AND LIFE

Remm's OpenClaw Personal OS

Speak naturally. Capture the right tasks. See today's plan. Remember every useful meeting. Build projects with Codex when inspiration hits.

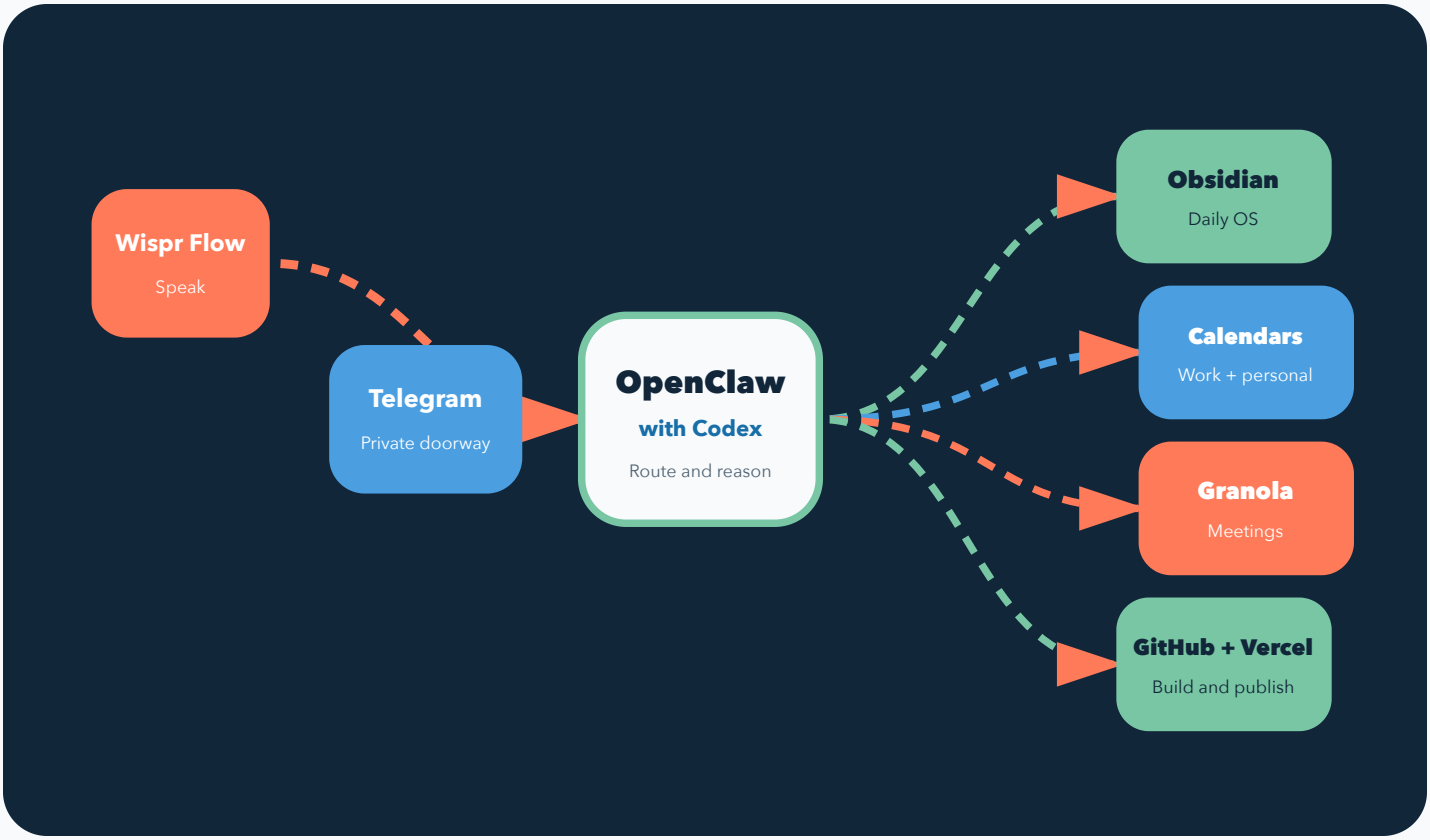
Designed for a personal MacBook and iPhone

Version 1.0 · August 2026 · HTML is the canonical guide

THE WHOLE SYSTEM ON ONE PAGE

Your voice becomes organized action

Wispr Flow helps you speak. Telegram gives you one private doorway. OpenClaw routes the request. Obsidian holds the durable record. Codex handles the reasoning.



One owner, one vault, one private bot. The public system code stays separate from the private runtime data.

WHAT KAZ SENDS REMM

One repository contains the complete package

Send the repository link. The HTML guide, both PDFs, installer, folder templates, skills, tests, and practice deployment are already inside it.

1

Repository

Codex clones the complete public package from GitHub.

2

Interactive HTML

`guide/index.html` is the detailed checklist and canonical setup guide.

3

Two PDFs

The full setup PDF teaches installation. The short landscape PDF supports the first conversation.

```
https://github.com/CraftYourDesires/openclaw-personal-os
```

- [ ] **Repository link received**
  This one link gives Codex every reusable setup file.
- [ ] **START-HERE.md located**
  Codex reads this file before installing or asking account questions.
- [ ] **Account ownership understood**
  Remm uses his own ChatGPT, Telegram, Google, GitHub, Vercel, Doppler, Granola, Wispr Flow, and Obsidian accounts.

Nothing needs to be copied out of the repository. Open the HTML locally after cloning. Attach either PDF to a Codex task when a visual reference is useful.

Create one Home folder, then let Codex work

The folder is named `Openclaw` and sits directly inside Remm's Mac Home folder. Codex creates everything underneath it.

Remm does this in Finder

1. Install Codex and sign in with paid ChatGPT.
2. Open Finder.
3. Press Shift, Command, and H.
4. Choose File, then New Folder.
5. Name the folder `Openclaw`.
6. Open that folder in Codex.

Codex creates this structure

```
~/Openclaw/
  system/      public repository
  runtime/     private personal data
    vault/     Obsidian
    workspace/ OpenClaw
    bin/       helper commands
    logs/      service logs
```

- [ ] **`~/Openclaw` created**
  The folder is inside Home, not Desktop, Downloads, Documents, or iCloud Drive.
- [ ] **Folder opened in Codex**
  Codex creates public code at `~/Openclaw/system` and private data at `~/Openclaw/runtime`.
- [ ] **Repository cloned into `system`**
  The exact final path is `~/Openclaw/system`.

Remm does not type Git commands. Paste the master prompt on page 18. Codex creates the folders, clones the repository, reads the files, and runs the installer.

ONE TIME CODEX SETTING

Give Codex Full access for this trusted setup

Full access lets Codex keep installing, configuring, testing, and using the network without stopping for routine command approval.

Turn it on

1. Press Command and comma to open Settings.
2. Choose General.
3. Under Permissions, turn on Full access.
4. Turn on Prevent sleep while running.

Select and save it

1. Return to the setup task.
2. Open the permissions control beneath the prompt box.
3. Choose Full access.
4. Ask Codex to preserve existing settings and make Full access the personal default.

- [ ] **Full access enabled**
  The mode is available in the task permissions menu.
- [ ] **Full access selected**
  The current task reports Full access before setup begins.
- [ ] **Prevent sleep enabled**
  The Mac can continue the local task while Remm steps away.
- [ ] **Personal default saved**
  Codex preserves existing settings and verifies Full access for future local tasks.

Trusted folders only. Full access can read or change files anywhere and use the network. Switch back to Ask for approval before opening unknown downloaded code. Account sign ins, macOS privacy prompts, secrets, destructive actions, and decisions outside the approved plan still require Remm.

BEFORE STARTING

What you need

Set aside 60 to 90 minutes. Most of the time is account sign in and permission screens. Codex handles the safe local installation.

1

Hardware

Personal MacBook, iPhone, internet connection, and administrator password.

2

Accounts

Paid ChatGPT, GitHub, Vercel, Doppler, Google, Telegram, Granola, Wispr Flow, and Obsidian Sync.

3

Time

Three short phases. Stop after any phase and resume later by running the doctor.

- [ ] **MacBook is plugged in**
  Keep it awake during setup and the first sync.
- [ ] **Paid ChatGPT account is ready**
  Codex and OpenClaw use the subscription sign in.
- [ ] **Google accounts are known**
  Have personal and work addresses ready. Work can wait if company OAuth blocks access.
- [ ] **Privacy notice reviewed**
  Meeting recording and full transcript retention must fit employer policy and local law.

Version one works while the Mac is awake. A server or always on Mac can come later if you want twenty four hour availability.

Phase 1 · Accounts and voice

Install the agentic toolkit

Codex runs the installer. It pins Node 24 so OpenClaw and QMD share one supported runtime.

```
cd "$HOME/Openclaw/system"
bash setup.sh
```

Command line tools

GitHub CLI, Vercel CLI, Codex CLI, OpenClaw, QMD, Doppler, Google Workspace CLI, Python, and Poppler.

Mac apps

Obsidian, Granola, and Wispr Flow. Open each app once after installation so macOS can request permissions.

- [ ] **Installer completed**
  It is safe to run the installer again. Personal vault files are preserved.
- [ ] **Codex CLI signed in**
  Run `codex login` and use the same paid ChatGPT account.
- [ ] **GitHub CLI signed in**
  Run `gh auth login`, choose GitHub.com, HTTPS, and browser sign in.
- [ ] **Vercel CLI signed in**
  Run `vercel login` and use the account that will own Remm's deployments.
- [ ] **Doppler signed in**
  Remm completes browser sign in. Codex creates the `openclaw-personal-os` project, selects the `dev` config, and verifies it.

References: OpenClaw Node guide, GitHub CLI, Vercel CLI.

Phase 1 · Private doorway

Create the Telegram bot

Telegram is the single place Remm talks to OpenClaw. The bot accepts direct messages only from Remm's numeric Telegram user ID.

1

Create

Open the verified **@BotFather**, send **/newbot**, and choose the bot's display name and username.

2

Store

Copy the bot token once. Paste it only into the private Doppler entry Codex opens.

3

Message

Open the new bot from BotFather, tap Start, and send the message **Hello**.

4

Pair

Tell Codex the message was sent. Codex reads the latest private update, confirms Remm's name, and stores the numeric sender ID.

- [ ] **Bot created with @BotFather**
  Confirm the username is exactly @BotFather before sharing a token.
- [ ] **Token stored in Doppler**
  Secret name: `TELEGRAM_BOT_TOKEN`. Never paste it into the repository.
- [ ] **Numeric owner ID stored**
  Codex confirms the sender name before storing `TELEGRAM_USER_ID`. This is the only accepted sender.
- [ ] **Private DM tested**
  Send “Create a test task for today” and verify one TaskNotes file appears.

> **Keep groups disabled initially.** Private DMs are easier to reason about and reduce accidental access.

Reference: OpenClaw Telegram setup.

Phase 1 · Speak naturally

Set up Wispr Flow

Wispr Flow turns speech into polished text inside Telegram. It does not replace OpenClaw. It makes the private chat fast enough to feel conversational.

Privacy Mode

Choose this when Remm wants less surrounding app context shared with Wispr. It can require slightly more explicit dictation.

Context Awareness

Choose this when Remm values better app specific corrections and is comfortable with the added context.

- [ ] **Wispr Flow signed in**
  Grant microphone and accessibility permissions when macOS asks.
- [ ] **Privacy choice recorded**
  Pick Privacy Mode or Context Awareness during onboarding.
- [ ] **Dictation shortcut practiced**
  Open the Telegram bot, hold the Wispr shortcut, speak, release, and review the text.
- [ ] **Real request tested**
  Say: “Remind me every Wednesday at nine to prepare the weekly update.”

Speak the outcome and the timing. Repeating requests become TaskNotes recurrence rules, so “every Wednesday at nine” keeps advancing after each completion.

Reference: Wispr Flow setup guide.

Phase 2 · Daily operating system

Open the Obsidian vault

The installer creates `~/Openclaw/runtime/vault`. Open that folder as an Obsidian vault, then connect it to Obsidian Sync.

B	**Bases** Required core plugin for TaskNotes views	D	**Daily Notes** Opens the current day
T	**Templates** Provides the daily note fallback	S	**Sync** Official paid Mac and iPhone sync
TN	**TaskNotes** One Markdown note per task	DV	**Dataview** Useful flexible vault queries
GC	**Google Calendar** Calendar views inside Obsidian	EX	**Excalidraw** Optional and disabled initially

- [ ] **Vault opened**
  Choose “Open folder as vault” and select `~/Openclaw/runtime/vault`.
- [ ] **Community plugins trusted**
  Review the listed plugins, then enable community plugins.
- [ ] **Obsidian Sync connected**
  Create or select a private remote vault, then connect the iPhone.
- [ ] **Test task appears**
  Create one TaskNotes task and confirm its file is in `TaskNotes/Tasks`.

References: Obsidian Sync setup, TaskNotes requirements.

Phase 2 · Calendars and optional email

Connect both calendars

The Google Workspace CLI gives OpenClaw scoped calendar access. The Obsidian calendar plugin gives Remm a visual calendar inside the vault.

Personal	Connect first. It counts as a successful setup even if work access is blocked.
Work	Connect second. If company OAuth blocks access, leave it pending and continue.
New events	Infer personal or work from context. Ask only when the destination is genuinely ambiguous.
Gmail	Optional and read only. Search and summarize only. Sending stays disabled in version one.

- [ ] **Google OAuth app created**
  Codex opens Google Cloud and guides one screen at a time. Choose Desktop app. Store the client ID and secret in Doppler, then let Codex run `personal-os google-credentials`.
- [ ] **Personal calendar connected**
  Run `personal-os google-auth personal EMAIL`. The browser asks for consent and the account is saved automatically.
- [ ] **Work calendar connected or marked pending**
  Do not block the system when company policy prevents OAuth.
- [ ] **Calendar routing tested**
  Create one personal test event, then one work event if available.
- [ ] **Optional Gmail choice recorded**
  If enabled, grant read only scopes and test one search.

Phase 2 · Today only

Your daily note is the home screen

Every morning, one focused note shows what matters today. It does not become a giant backlog.

Today at a glance

Work and personal events appear with source labels.

Due tasks

Only open tasks scheduled or due today.

Granola review

Meetings with proposed actions waiting for approval.

Reminders and notes

A short scratch space for today's reminders and thinking.

```
personal-os daily
```

- [ ] **Today's note generated**
  Confirm the file exists in `Daily` with today's date.
- [ ] **Calendar labels are correct**
  Personal and Work appear beside the right events.
- [ ] **Due task appears**
  Create a task due today and refresh the note.
- [ ] **Note reaches iPhone**
  Open Obsidian on iPhone and verify the same note through Sync.

> **Rule of thumb:** the daily note tells you what deserves attention today. TaskNotes and Vault Recall hold everything else.

Phase 3 · Meetings and memory

Granola captures. Codex proposes. Remm approves.

The signed in Granola desktop session supplies meeting notes and full transcripts. Codex creates a review packet, not automatic commitments.

1
Granola records the meeting.

→

2
The hourly sync writes the full transcript.

→

3
Codex proposes actions and decisions.

Review state

Proposed actions remain checkboxes in the meeting note. They appear in the daily note.

Approved state

Only selected action numbers become TaskNotes files. Unselected actions remain visible.

- [ ] **Granola signed in**
  Open the desktop app and finish its account setup.
- [ ] **Controlled meeting recorded**
  Use a short test meeting with one decision and three action items.
- [ ] **Full transcript imported**
  Verify the meeting note contains the complete Transcript section.
- [ ] **Review packet created**
  Confirm summary, proposed actions, decisions, topics, and status review.
- [ ] **Selected actions approved**
  Approve actions 1 and 3. Confirm two task files and one remaining proposal.

Durability note: the local Granola desktop authentication method is convenient and free, but an app update can change it. The setup doctor reports this failure clearly.

References: Granola transcription, Granola export options.

Phase 3 · Vault Recall

Ask your history a normal question

Vault Recall searches notes, TaskNotes, meeting transcripts, and OpenClaw memory. QMD runs locally and Codex synthesizes the retrieved evidence.

Decision recall

"What did Jordan decide about the launch date?"

Open loops

"What am I waiting on from the product team?"

Person recall

"What has Alex said about the new workflow?"

Date recall

"What happened in meetings last Thursday?"

```
personal-os index
personal-os recall "launch decision and checklist owner" 8
```

Required agent abilities: custom Vault Recall, bundled `gog`, reviewed capture and meeting workflows, plus `gh` and `vercel` for agentic projects.

- [ ] **QMD collection indexed**
  The `personal-os` collection appears in QMD status.
- [ ] **Meeting answer cites evidence**
  Ask about the controlled meeting and verify the note name and date.
- [ ] **Transcript wins conflicts**
  A conflicting draft summary must be corrected with transcript evidence.
- [ ] **Missing evidence is honest**
  Ask about a made up project and verify the answer says it was not found.

Reference: QMD project.

Phase 3 · Build and publish

GitHub and Vercel become normal tools

GitHub CLI lets Codex create repositories, branches, commits, and pull requests. Vercel CLI lets Codex link and deploy web projects from the terminal.

1

Own the accounts

Remm signs in to GitHub and Vercel. Projects belong to Remm, not to the template author.

2

Deploy the status page

The `status` folder contains a nonsecret first project that proves GitHub to Vercel works.

```
cd "$HOME/Openclaw/system/status"
vercel
```

- [ ] **Private practice repository created**
  Use this public template to create a private Remm owned project.
- [ ] **Status page committed**
  The page contains no names, calendars, tasks, transcripts, tokens, or private URLs.
- [ ] **Vercel deployment succeeds**
  Open the public URL and confirm the page loads.
- [ ] **Agentic CLI test succeeds**
  Ask Codex to change one sentence, commit it, push it, and deploy the update.

SECURITY AND PRIVACY

Useful access, narrow boundaries

The goal is not arbitrary control of the Mac. The goal is a small set of reliable actions with clear ownership and review.

Telegram	One numeric owner ID. Direct messages only. Groups disabled initially.
Files	Codex setup access covers `~/Openclaw`. OpenClaw runtime execution stays scoped to `~/Openclaw/runtime`.
Commands	Host execution defaults to an allowlist and asks on a miss.
PIN	Supported app installation requires a short lived authorization. The PIN hash lives in Doppler.
Secrets	Tokens live in Doppler and enter the gateway process at runtime. Repositories contain placeholders only.
Meetings	Full transcripts remain in the private vault indefinitely unless Remm deletes them.

```
Doppler names: TELEGRAM_BOT_TOKEN, TELEGRAM_USER_ID, OPENCLAW_GATEWAY_TOKEN, GOOGLE_CLIENT_ID,
GOOGLE_CLIENT_SECRET, PERSONAL_OS_PIN_HASH
```

- [ ] **PIN configured**
  Run `personal-os set-pin` in the terminal. Enter the PIN privately.
- [ ] **Secret audit clean**
  Run the OpenClaw secrets audit and confirm no plaintext findings.
- [ ] **Unknown sender denied**
  Test with a second Telegram account only if one is safely available.
- [ ] **Obsidian Sync verified**
  Open a test note on Mac and iPhone before trusting the vault with important work.

References: OpenClaw exec approvals, OpenClaw secrets management.

VERIFICATION

Run one setup doctor

The doctor is the resume point. Run it after each phase. Fix the first required item, then run it again.

```
personal-os doctor
```

Required checks

Node 24, all CLIs, apps, vault, required plugins, Codex sign in, GitHub sign in, Vercel sign in, Doppler, Google account, Granola session, and QMD.

Optional checks

Work calendar and read only Gmail can remain pending without blocking the successful personal setup.

- [ ] **All required checks pass**
  No required line is marked FIX.
- [ ] **Background services started**
  Run `personal-os services-start`, then verify with `personal-os services-status`.
- [ ] **Voice to task passes**
  Dictate one due task and see it in today's note.
- [ ] **Voice to calendar passes**
  Dictate one event and verify the selected calendar.
- [ ] **Meeting review passes**
  Import, propose, preview, approve, and verify selected tasks.
- [ ] **Vault Recall passes**
  Answer one decision question with transcript evidence.
- [ ] **GitHub to Vercel passes**
  The nonsecret status page is live.

> **Common fix:** if QMD or OpenClaw reports a native module error, run `node -v`. It should show version 24, then reinstall QMD under that same runtime.

COPY THIS INTO REMM'S CODEX

The master onboarding prompt

Open `~/Openclaw` in Codex and paste the prompt below. Codex creates the two folders, clones the repository, installs safe local pieces, and pauses for human account steps.

```
Goal: complete my OpenClaw Personal OS setup on this Mac from a blank starting point.

Success means:
* My public system repository is located at ~/Openclaw/system.
* My private runtime is located at ~/Openclaw/runtime.
* Codex Full access is enabled, selected for this task, and saved as the default for future local tasks on this Mac.
* Prevent sleep while running is enabled.
* Install the safe local tools and apps through setup.sh.
* Guide me through paid ChatGPT with Codex, GitHub CLI, Vercel CLI, Doppler, Telegram BotFather, OpenClaw, Wispr Flow, Obsidian Sync, Granola, two Google calendars, and optional read only Gmail.
* Configure Obsidian core plugins Bases, Daily Notes, Templates, and Sync.
* Install and verify TaskNotes, Dataview, and Google Calendar. Keep Excalidraw optional and disabled initially.
* Use my personal calendar as the required success path. Treat work calendar OAuth as optional when my company blocks it.
* Configure my private Telegram bot for my numeric owner ID only.
* Route routine task, reminder, and calendar actions immediately when details are clear. Ask one short question when the destination or timing is ambiguous.
* Require the configured PIN before supported high risk system actions.
* Import Granola meetings through the signed in desktop session, retain full transcripts, create Codex review packets, and create real tasks only after I approve selected action numbers.
* Build today's focused daily note and verify it on my iPhone through Obsidian Sync.
* Install and verify Vault Recall with QMD under Node 24.
* Create a private practice repository owned by me and deploy the nonsecret status page through Vercel.
* Run personal-os doctor after each phase and resolve every required FIX item.

Stop when: personal-os doctor reports that every required check passes and the controlled voice, task, calendar, Granola, recall, GitHub, and Vercel tests succeed.

Constraints: Preserve existing files. Keep private data in ~/Openclaw/runtime. Keep reusable public code in ~/Openclaw/system. Store secrets through Doppler. Use my own accounts. Do not pause for routine Codex command approvals after Full access is active. Pause for every personal sign in, macOS permission, private token, paid plan choice, PIN entry, destructive action, or material decision outside this approved plan.

Create ~/Openclaw if it is missing. Clone https://github.com/CraftYourDesires/openclaw-personal-os into ~/Openclaw/system if that folder is missing. Read START-HERE.md, README.md, DECISIONS.md, setup.sh, and guide/index.html before installing.

First verify that this task reports Full access and that Prevent sleep while running is enabled. Preserve my existing Codex settings and make Full access the default for my future local tasks using the current Codex configuration format. Then run the safe setup steps yourself. Give me one short human instruction at a time only when I must sign in, approve a macOS privacy prompt, enter a private value, or make a material decision. Continue routine installation and verification without asking for Codex command approval. Run personal-os doctor after each phase and use its first required FIX item as the next step.
```

> **Done means verified.** The doctor, controlled meeting, recall question, and status deployment prove the connections, not just the presence of installed apps.

OFFICIAL REFERENCES

Sources and ownership

Use these pages when a setup screen changes. Software moves quickly, so prefer the current official documentation over screenshots.

- OpenClaw Node requirements
- OpenClaw OpenAI and Codex auth
- OpenClaw Telegram channel
- OpenClaw exec approvals
- Codex permission modes
- OpenClaw secrets management
- Wispr Flow setup
- Granola transcription
- Granola export
- Obsidian Sync
- TaskNotes
- QMD
- GitHub CLI
- Vercel CLI

Remm owns

Accounts, tokens, vault, transcripts, repositories, deployments, calendar data, and all private project copies.

The public template owns

Only reusable setup code, empty vault structure, guide assets, tests, and nonsecret status page.